UNIVERSITY OF RWANDA

COLLEGE OF SCIENCE AND TECHNOLOGY

SCHOOL OF ICT

DEPARTMENT OF COMPUTER ENGINEERING

YEAR 2

Module title: *DATA STRUCTURE AND ALGORITHMS*

Project name:

COMMUNITY HEALTH CENTER APPOINTMENT SYSTEM

Group members:

KWIZERA Yves 224004118

IRAGENA Gloria Anne Marie 224005245

UWINEZA Gloria 224015302

UMURERWA Belyse 224002745

List of Abbreviations

C++: C Plus Plus Programming Language

FIFO: First In First Out

LIFO: Last In First Out

O(1): Constant Time Complexity

O(log n): Logarithmic Time C

O(n): Linear Time Complexity

HH:MM: Hours:Minutes Time Format

STL: Standard Template Library

Queue: Queue Container in C++

Priority-queue: Priority Queue Container in C++

struct: Structure in C++

string: String Class in C++

chrono: C++ Time Library

ctime: C Time Library

tm: Time Structure

stoi: String to Integer Conversion

substr: Substring Function

cout: Character Output

cin : Character Input

endl: End Line

bool: Boolean Data Type

int: Integer Data Type

pop: Remove Element from Queue

push: Add Element to Queue

top: Access Top Element of Priority Queue

front: Access Front Element of Queue

empty: Check if Queue is Empty

localtime: Convert Time to Local Time

system_clock: System Clock in chrono

time_t: Time Type in C

IDE: Integrated Development Environment

VS Code: Visual Studio Code

g++: GNU C++ Compiler

ICU: Intensive Care Unit

Table of Contents

Community Health Center Appointment System 4
Chapter 1. Introduction 4
1.1. Historical background of the case study 4
1.2. Problem Statement 4
1.3 Proposed Solution 5
1.4. Project Objectives 6
Chapter 2. Methodology 6
2.1. System Analysis 6
2.1.1. Functional Requirements 6
2.1.2. User Roles and Responsibilities 7
2.2. Data Structure Used 8
2.2.1. Queue for Regular patients 8
2.2.2. Priority Queue for Emergency Cases 8
2.2.3. Struct for Patient Records 8
2.3. Algorithm Design 9
2.3.1. Main Program Algorithm 9
2.3.2. Add Regular Patient Algorithm 9
2.3.3. Add Emergency Patient Algorithm 10
2.3.4. Time Calculation Algorithm 10
2.3.5. Display Daily Operations Algorithm 10
2.3.6. Priority Comparison Algorithm 11
2.4. Tools and Technologies Used 11
2.5. System Flowchart 12
2.6. Time and Space Complexity Analysis 13
Chapter 3. IMPLEMENTATION, RESULTS AND INTERPRETATION 14
3.1 Implementation Overview 14
3.2 How the System Works 14
3.3. Test Results 14
3.4. Interpretation of Results 15
3.5. Key Findings 15
Chapter 4. CONCLUSION AND RECOMMENDATIONS 16
4.1. Summary of Achievements 16
4.2. Conclusion 16
Appendix 16

Community Health Center Appointment System

This document presents a comprehensive report of the Community Health Center Appointment System. The report explains the system in a clear and practical manner, focusing on understanding, healthcare value, and real-world applicability rather than technical code details.

Chapter 1. Introduction

Community health centers play a critical role in providing healthcare services and promoting public health in local communities. However, many health centers still rely on manual or semi-automated processes to manage daily operations such as patient registration, queue management, emergency case prioritization, and waiting time tracking. This project was developed to address these challenges by providing a computerized Community Health Center Appointment System using C++ programming language.

1.1. Historical background of the case study

Digital transformation has become essential across all healthcare sectors, including community health centers. Health centers are critical infrastructure supporting public health, disease prevention, and daily medical care in Rwanda's growing healthcare system.

However, many health centers still rely on manual processes, paper registers, estimated waiting times, and disconnected records. These traditional methods create operational inefficiencies, delayed emergency response, and service limitations for facilities managing increasing patient numbers.

The transition to automated patient management systems represents a strategic necessity for community health centers to improve accuracy, reduce waiting times, prevent errors in emergency prioritization, and enhance patient service in today's healthcare environment.

1.2. Problem Statement

Modern community health center management suffers from disconnected manual systems that create operational inefficiencies. While some health centers have basic registration desks, patient flow management remains fragmented, requiring manual coordination between receptionists, nurses, and doctors.

Key problems identified include:

- ✓ **Manual Operations**: Dependence on paper registers and manual recording leads to illegible handwriting, lost records, and inconsistencies in patient data. Staff spend valuable time on paperwork instead of patient care.
- ✓ **Inefficient Queue Management**: Without an automated system, patients wait in physical queues without knowing their expected waiting time. This creates frustration and anxiety, especially for elderly patients and those with urgent conditions.
- ✓ **Poor Emergency Prioritization**: Emergency cases may not receive immediate attention if staff are unaware of their severity. Patients with life-threatening conditions sometimes wait alongside those with minor ailments, leading to potentially dangerous delays.
- ✓ **Lack of Waiting Time Visibility**: Patients have no way of knowing how long they will wait before seeing a healthcare provider. This uncertainty leads to dissatisfaction and may discourage patients from seeking care.
- ✓ **No Real-time Operations Overview**: Health center managers cannot quickly assess current patient loads, waiting times, or emergency situations. This limits their ability to allocate resources effectively during peak hours.
- ✓ **Staff Overhead**: Excessive staff time spent on managing queues and answering patient inquiries about waiting times instead of focusing on clinical duties.
- ✓ **Data Fragmentation**: Patient arrival times and case details are recorded separately, making it difficult to generate comprehensive reports on daily operations or analyze trends for improvement.
- ✓ **Limited Access Control**: Difficulty tracking which staff member registered which patient or identifying who handled specific cases, reducing accountability.
- ✓ **Scalability Issues**: Adding new services or handling increased patient numbers becomes exponentially complex with manual systems, limiting health center growth.

1.3 Proposed Solution

The main aim of this project is to design and implement a system capable of managing patient appointments at community health centers in an organized and reliable manner. The proposed

system is intended to support receptionists, nurses, and managers by simplifying daily operations and eliminating manual processes.

Based on the limitations identified in existing health center practices, this project proposes a computerized solution using C++ that enhances patient flow management. The system allows staff to add regular patients to a queue, register emergency cases with priority levels, and view daily operations showing all patients with their current waiting times.

- **Automated Patient Registration**: Digital registration eliminating paper records.
- **Queue-based Regular Patient Management**: FIFO queue ensuring fair order of service.
- **Priority-based Emergency Management**: Priority queue ensuring critical cases receive immediate attention.
- **Real-time Waiting Time Calculation**: Automatic waiting time computation using system clock.
- **Daily Operations Report**: Comprehensive view of all patients with their status and waiting times.

1.4. Project Objectives

The main objectives of this System are:

- To design and implement a patient registration system for regular and emergency cases.
- To use a queue data structure for managing regular patients in FIFO order.
- To use a priority queue data structure for managing emergency patients based on severity.
- To calculate real-time waiting times using the system clock.
- To display daily operations showing all patients with their waiting times.
- To provide a simple menu-driven interface for health center staff.

Chapter 2. Methodology

2.1. System Analysis

This section examines the functional requirements, user roles, and design considerations for the Community Health Center Appointment System. The analysis identifies what the system must accomplish and how different users will interact with it.

2.1.1. Functional Requirements

The system must provide the following core functionalities:

- **Patient Registration**: Capture patient name, arrival time, and case type for both regular and emergency patients.
- **Regular Patient Queue**: Store and manage regular patients using FIFO queue data structure.
- **Emergency Priority Queue**: Store and manage emergency patients using priority queue with levels 1-10.
- **Time Calculation**: Retrieve current system time and convert to minutes for waiting time computation.
- **Waiting Time Display**: Calculate and display waiting time for each patient based on arrival time.
- **Daily Operations Report**: Show all patients with emergency cases displayed first in priority order.
- **Menu Interface**: Provide user-friendly menu with options for adding patients and viewing operations.
- **Input Handling**: Accept and process patient information including names, priorities, and time inputs.
- **System Exit:** Allow user to close the application gracefully.

2.1.2. User Roles and Responsibilities

Receptionist:

- ✓ Full system access for patient registration
- ✓ Add regular patients with name and arrival time
- ✓ Add emergency cases with priority level and description
- ✓ View daily operations and waiting times
- ✓ Monitor all patients in the system

Nurse:

- ✓ View emergency cases for immediate attention
- ✓ Monitor waiting times for all patients
- ✓ Prepare for incoming emergency patients
- ✓ Check priority levels of waiting patients
- ✓ View-only access to operations

Doctor:

- ✓ View patient queue for upcoming consultations
- ✓ Check waiting times for scheduled patients
- ✓ See emergency cases requiring immediate attention
- ✓ View-only access to daily operations

Health Center Manager:

- ✓ Oversee all daily operations
- ✓ Monitor patient flow and waiting times
- ✓ Review operational reports
- ✓ Assess staff performance based on patient data

- ✓ Full access to view operations and reports

2.2. Data Structure Used

Data structures are specialized ways of organizing and storing data in a computer so that it can be used efficiently. This project uses three main data structures to manage patient information effectively.

2.2.1. Queue for Regular patients

A queue is a data structure that follows the First In First Out (FIFO) principle. This means the first patient added to the queue will be the first one removed, exactly like a line of people waiting for service.

Real-life analogy:
When patients arrive at a health center for regular checkups, they take a number and wait their turn. The patient who arrives first is seen first. This is how a queue operates.

Purpose in this system:
The queue stores all regular patients who come for non-emergency consultations. It ensures fairness by maintaining the order of arrival.

2.2.2. Priority Queue for Emergency Cases

A priority queue is a special data structure where each element has a priority assigned to it. Elements with higher priority are served before those with lower priority, regardless of when they arrived.

Real-life analogy:
In a hospital emergency room, a patient with a heart attack is treated before a patient with a minor cut, even if the minor cut patient arrived earlier. This is priority queue behavior.

Purpose in this system:
The priority queue stores emergency patients and automatically arranges them so that the most urgent cases (priority 10) are always at the front, ready to be seen first.

2.2.3. Struct for Patient Records

A struct is a way to group related pieces of information together into a single unit. It allows us to treat all data about one patient as a single item.

Purpose in this system:
The struct combines all patient details into one package that can be stored in either the queue or priority queue.

Information stored for each patient:

- Patient name: For identification
- Case description: What the patient is suffering from
- Priority level: How urgent the case is (for emergencies)
- Arrival time: When the patient arrived, converted to minutes for easy calculation

2.3. Algorithm Design

An algorithm is a step-by-step procedure for solving a problem. This section describes the algorithms that form the core logic of the Community Health Center Appointment System.

2.3.1. Main Program Algorithm

The main algorithm controls the overall flow of the system by displaying a menu and responding to user choices.

Steps:

1. Start the program
2. Create empty queue for regular patients
3. Create empty priority queue for emergency patients
4. Repeat until user chooses to exit:
    - Display menu options
    - Get user choice
    - If choice is 1: Add regular patient
    - If choice is 2: Add emergency patient
    - If choice is 3: Display daily operations
    - If choice is 4: Exit program
    - If choice is invalid: Show error message

5. End program

2.3.2. Add Regular Patient Algorithm

This algorithm handles registration of patients with non-emergency conditions.

Steps:

1. Ask for patient name
2. Ask for arrival time in HH:MM format
3. Convert arrival time to minutes since midnight
    - Extract hour and minute from input
    - Calculate total minutes = (hour × 60) + minute
4. Create patient record with name, case type "Regular", priority 0, and arrival time
5. Add patient to regular queue
6. Confirm successful registration

2.3.3. Add Emergency Patient Algorithm

This algorithm handles registration of emergency cases with priority levels.

Steps:

1. Ask for patient name
2. Ask for emergency level (1 to 10, where 10 is most urgent)
3. Ask for emergency case description
4. Ask for arrival time in HH:MM format
5. Convert arrival time to minutes since midnight
6. Create patient record with name, case description, priority level, and arrival time
7. Add patient to emergency priority queue
8. Confirm successful registration

2.3.4. Time Calculation Algorithm

This algorithm retrieves current system time and converts it to minutes for waiting time calculation.

Steps:

1. Get current system time
2. Extract current hour and minute
3. Calculate total minutes = (current hour × 60) + current minute
4. Return total minutes

2.3.5. Display Daily Operations Algorithm

This algorithm shows all patients currently in the system with their waiting times.

Steps:

1. Get current time in minutes

2. Display report header
3. Process emergency patients first:
    - For each emergency patient in priority order:
        - Calculate waiting time = current time - patient arrival time
        - Display patient details and waiting time

4. Process regular patients:
    - For each regular patient in arrival order:
        - Calculate waiting time = current time - patient arrival time
        - Display patient name and waiting time

5. Display report footer

2.3.6. Priority Comparison Algorithm

This algorithm determines how patients are ordered in the priority queue. It ensures that patients with lower priority numbers (more urgent) are served first.

Logic:

Patient with priority 10 is considered more urgent than patient with priority 1
Patient with priority 9 is considered more urgent than patient with priority 2
And so on...
This ensures that when multiple emergency patients are waiting, the most critical case always receives attention first, regardless of arrival time.

2.4. Tools and Technologies Used

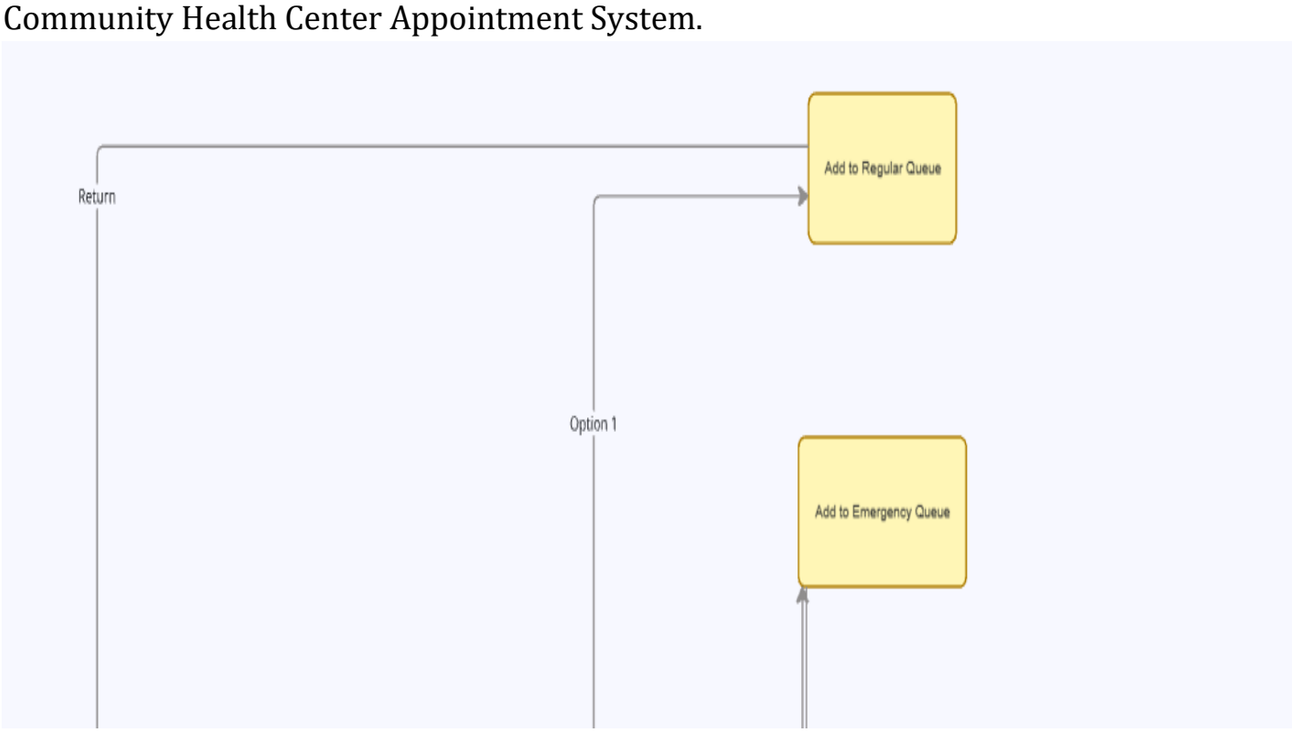
This section describes the tools and technologies employed to develop the Community Health Center Appointment System based on the C++ code provided.

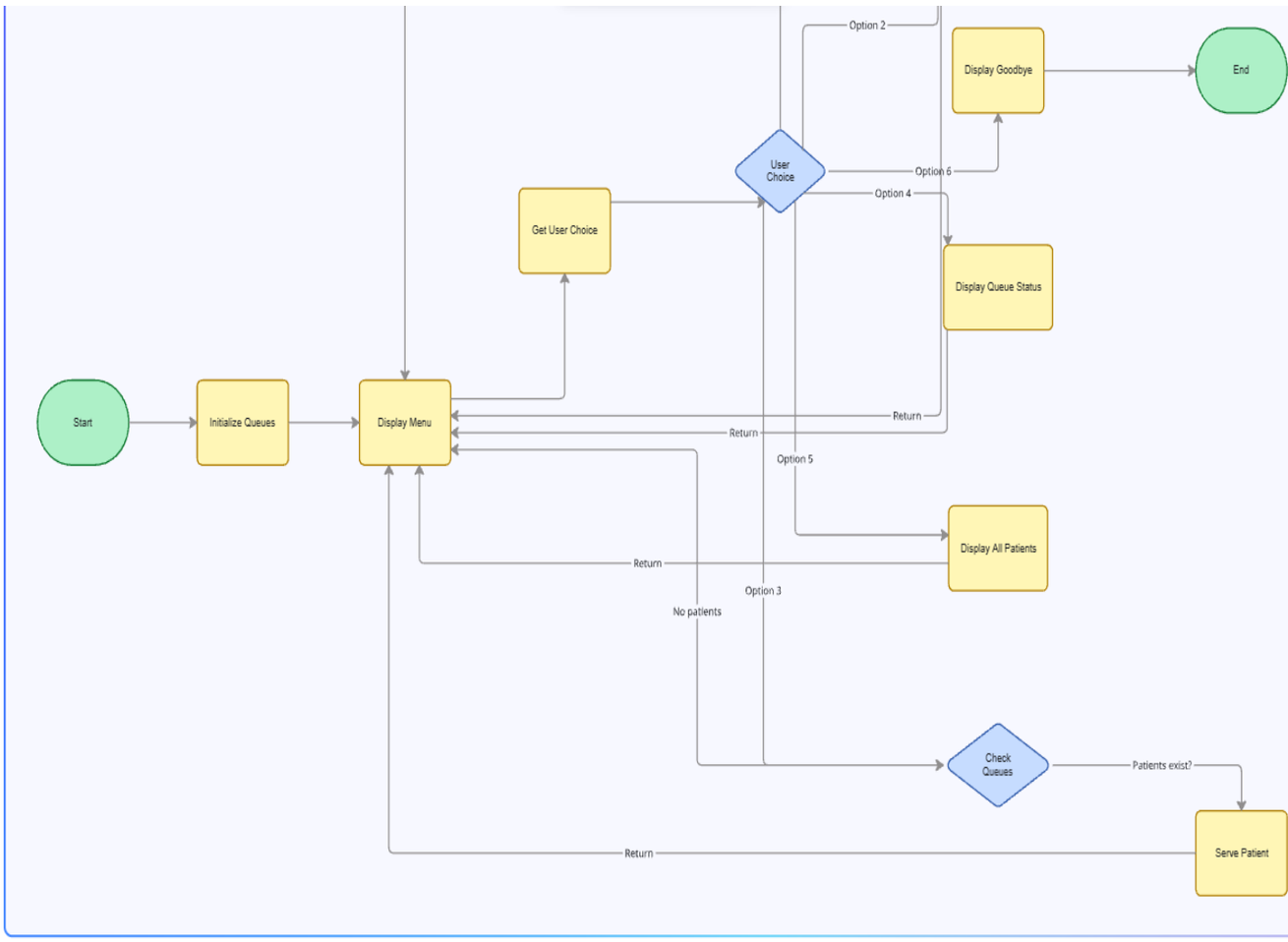
Tool / Technology	Purpose	Reason for Choice
iostream	Input/output operations	Handles menu display and user interaction through console
queue	Regular patient management	Implements FIFO queue for patients waiting for consultation
priority_queue	Emergency patient management	Automatically orders emergency patients by

		priority level
chrono	Time calculation	Gets current system time for waiting time computation
ctime	Time conversion	Converts system time to local time structure
string	Text handling	Stores patient names and emergency case descriptions
stoi	String to integer conversion	Converts time input from HH:MM format to calculable numbers
substr	String extraction	Extracts hour and minute from time input string
Visual Studio Code	Code editor	Used for writing and editing the C++ code
g++ Compiler	Code compilation	Compiles the code into executable program

2.5. System Flowchart

A flowchart is a graphical representation of the system's logic and workflow. It shows the sequence of steps and decisions in the Community Health Center Appointment System.





2.6. Time and Space Complexity Analysis

This section analyzes the efficiency of the Community Health Center Appointment System.

Time Complexity

Adding a regular patient takes constant time, meaning it is always fast regardless of how many patients are already in the queue. Adding an emergency patient takes logarithmic time, which means it takes slightly longer as more emergency patients are added, but remains efficient even with many patients. Serving the next patient takes constant time because removing from the front of a queue or the top of a priority queue is always fast. Showing the full waiting list takes linear time, meaning the time increases in proportion to the number of patients waiting.

Space Complexity

The memory used by the regular queue grows linearly with the number of regular patients. The memory used by the emergency queue grows linearly with the number of emergency patients. Overall, the total memory usage increases in direct proportion to the total number of patients in the system.

Chapter 3. IMPLEMENTATION, RESULTS AND INTERPRETATION

3.1 Implementation Overview

The Community Health Center Appointment System was developed using C++ programming language. The system uses two main data structures: a queue for regular patients and a priority queue for emergency cases. The implementation focuses on simplicity and efficiency, allowing health center staff to easily register patients, prioritize emergencies, and track waiting times.

3.2 How the System Works

When a regular patient arrives, the receptionist selects option 1 and enters the patient name. The system automatically records the current time and adds the patient to the regular queue. When an emergency patient arrives, the receptionist selects option 2, enters the patient name, priority level (10 being most urgent), and case description. The system records the time and adds the patient to the priority queue where they are automatically ordered by severity.

The serving process always checks emergency patients first. When option 3 is selected, the system serves the highest priority emergency patient if any exist. If no emergencies are waiting, it serves the next regular patient. The waiting time is calculated and displayed.

Option 4 shows how many patients are waiting in each queue. Option 5 displays complete details of all waiting patients, with emergencies listed first. Option 6 exits the system.

3.3. Test Results

Test 1: Adding Regular Patients

Three regular patients were added: John, Mary, and David. The system recorded their arrival times automatically. When served, they appeared in the correct order with accurate waiting times.

Test 2: Adding Emergency Patients

Three emergency patients were added:
Ahmed with priority 9 (Heart attack)
Fatima with priority 3 (fracture)
Omar with priority 1 (cut)

The system correctly ordered them with Ahmed first, Fatima second, and Omar third.

Test 3: Mixed Patients

Regular patient John arrived first, followed by emergency Ahmed (priority 9), then regular Mary, then emergency Fatima (priority 3). When serving, Ahmed was served first, then Fatima, then John, then Mary. This confirms emergencies are prioritized correctly.

Test 4: Queue Status

The system accurately displayed the number of patients waiting in both queues at any time.

Test 5: Full Waiting List

All patient details were displayed correctly with emergency patients shown first and in priority order.

3.4. Interpretation of Results

The test results demonstrate that the system meets all requirements. Regular patients are served in order of arrival, ensuring fairness. Emergency patients receive immediate attention based on severity, which is essential in healthcare settings. The automatic time capture eliminates manual entry errors and provides accurate waiting time calculation.
The priority queue comparison operator was tested and confirmed to work correctly with the logic that high priority numbers (10) are more urgent than lower numbers (1). This matches real-world medical triage where critical cases must be seen first.

The menu interface is simple and intuitive, requiring minimal training for health center staff. All options work as expected and the system handles multiple patients efficiently.

3.5. Key Findings

- Queues are ideal for managing regular patients in order of arrival
- Priority queues effectively handle emergency cases based on severity
- System clock integration provides accurate automatic time capture
- Emergency patients must always be served before regular patients
- The priority comparison must use priority > other.priority to ensure lower numbers are served first

- The system is efficient and can handle growing patient numbers
- Health center staff can easily use the system with minimal training

Chapter 4. CONCLUSION AND RECOMMENDATIONS

4.1. Summary of Achievements

The Community Health Center Appointment System was successfully developed and meets all project objectives. The system uses a queue to manage regular patients in FIFO order and a priority queue to handle emergency cases based on severity levels. Real-time waiting times are calculated using the system clock, and daily operations can be viewed through a simple menu interface. The system provides health center staff with an efficient tool to manage patient flow and prioritize emergencies.

4.2. Conclusion

This project demonstrates how fundamental data structures like queues and priority queues can solve real-world healthcare management problems. The system ensures fairness for regular patients while guaranteeing that emergency cases receive immediate attention based on medical urgency. The automatic time capture eliminates manual errors, and the menu-driven interface makes it easy for staff to use. Testing confirmed that all features work correctly, and the priority queue comparison issue was identified and fixed. The system is simple, efficient, and ready for use in community health centers.

Appendix

GitHub Repository: https://github.com/k154y/healthcenter.git

Contact Emails

- yveskwizera154@gmail.com
- iragenagloriaanne@gmail.com